

전기전자통신공학부

네트워크프로그래밍및실습 결과보고서

- UDP 통신 방식의 등록 서버와 TCP 통신을
통한 유저간 채팅 시스템 구현 -

팀 원 : 류황민

강의교수 : 이재오 교수님

제 출 일 : 2025. 06. 22.

목 차

I. 서론	01
1. 작품선정 배경 및 필요성	01
2. 연구 목적	01
3. 역할 분담 및 팀 소개	01
II. 본론	02
1. 연구 내용	02
2. 문제점 및 해결방안	02
3. 연구과제 구현방법	03
III. 결론	10
1. 연구 결과	10

I. 서론

1. 작품선정 배경 및 필요성

현대 사회에서 네트워크와 분산 시스템의 중요성은 나날히 커지고 있습니다. 특히 여러 모바일 시스템들이 효율적으로 정보를 동적으로 공유하고, 다양한 서비스에 원활하게 접속할 수 있는 환경을 구현하는 것은 산업 현장과 학계 모두에서 중요한 과제로 인식되고 있습니다. 이에 본 프로젝트에서는 등록 서버를 활용해 모바일 시스템 간의 동적 연결 정보를 효율적으로 관리할 수 있는 시스템을 구현하고자 하였습니다.

분산 환경에서는 각 클라이언트와 서버가 상대방의 연결 정보를 직접적으로 알지 못하는 경우가 많습니다. 이러한 상황에서 등록 서버와 같은 중개 시스템을 도입하면, 각 시스템이 자신의 정보를 효율적으로 등록하고, 필요한 경우 상대방의 정보를 안전하게 획득할 수 있어 시스템의 확장성과 안정성이 크게 향상됩니다.

이러한 구조는 클라우드 서비스, 채팅 서버, IoT 시스템 등 다양한 실제 서비스에 폭넓게 적용되고 있습니다. 본 프로젝트를 통해 이러한 시스템을 직접 구현해봄으로써 실습 경험을 쌓고자 하였으며, 그 중에서도 일상적으로 많이 사용되고 구현 난이도도 적절한 채팅 시스템을 주제로 선정하였습니다.

2. 연구 목적

본 프로젝트의 목적은 응용 서비스를 직접 개발하고 이를 네트워크 서비스와 연동함으로써, 분산 환경에서의 정보 교환 및 동적 연결의 효율성을 직접 경험하고 검증하는 것입니다. 특히, 등록 서버를 기반으로 클라이언트들이 자신을 등록하고 서로의 정보를 안전하게 획득하며, 실시간으로 연결할 수 있는 구조를 구현함으로써 네트워크 응용 서비스의 실제 동작 원리와 장점을 체험하는 것이 주요 목표입니다.

또한, 본 프로젝트를 통해 채팅 시스템과 같은 응용 서비스를 네트워크 서비스와 효과적으로 결합함으로써, 확장성(다수의 클라이언트가 효율적으로 접속하고 정보 교환이 가능한 구조 설계), 유연성(동적으로 접속·등록·삭제가 가능한 서비스 제공), 실시간성(실시간으로 필요한 정보를 획득하고 즉각적으로 서비스를 제공하는 경험) 등을 실제로 구현하고 분석해보고자 합니다. 이를 통한 분산 시스템 환경에서의 응용 서비스 개발 역량을 강화하고, 실무에서 요구되는 네트워크 설계 및 구현 능력을 키우는 것을 목표입니다.

3. 역할 분담 및 팀 소개

소속학과	역할	성명	연락처
정보통신공학부	총괄	류황민	010-8953-4438

II. 본 론

1. 연구 내용

본 연구에서는 분산 네트워크 환경에서 클라이언트가 등록 서버를 통해 동적으로 자신의 정보를 등록하고, 다른 유저의 정보를 조회하여 TCP를 통한 P-to-P 채팅 서비스에 연결하는 구조를 설계 및 구현하였습니다. 주요 구현 내용은 아래와 같습니다.

1) 등록 서버 구현

모바일 시스템이 자신의 ID와 연결 정보를 등록하고, 서버가 이를 효율적으로 관리할 수 있도록 UDP 기반 등록 서버를 개발하였습니다. 서버는 등록, 조회, 수정, 삭제 등 네 가지 주요 기능을 제공하며, 접속 중인 유저 목록 및 특정 유저의 정보를 안전하게 관리할 수 있습니다.

2) 클라이언트 UI 및 등록 서버 등록

사용자는 클라이언트 프로그램을 통해 자신의 정보를 등록하고, 등록된 다른 유저의 정보를 조회하거나 자신의 정보를 수정·삭제할 수 있습니다. 클라이언트는 네트워크 기능과 사용자 인터페이스(UI)를 분리하여 설계하였으며, 사용자의 명령에 따라 서버와 통신을 수행합니다.

3) 응용 서비스(채팅 시스템) 구현

등록 서버를 통해 각 사용자가 필요한 상대방의 연결 정보를 획득한 후, 클라이언트 간에는 TCP 통신을 이용한 직접 채팅 기능을 구현하였습니다. 이를 통해 사용자는 등록 서버에서 받은 정보를 바탕으로 상대방과 안정적으로 실시간 채팅을 주고받을 수 있도록 하였습니다.

이를 통해, 네트워크 분산 환경에서 등록 서버가 어떤 역할을 하는지, 그리고 응용 서비스가 실제로 어떻게 네트워크 기능과 결합되어 동작하는지 구체적으로 실습하고 분석하였습니다.

2. 문제점 및 해결방안

프로그램을 설계하고 구현하는 과정에서 아래와 같은 문제들이 발생하였습니다.

1. UDP/TCP 전송 시 한글이 깨지는 현상이 발생

이 문제의 원인으로는 입출력 스트림 간 인코딩(예: UTF-8) 처리의 불일치, 혹은 일부 환경에서의 패킷 손실, 버퍼 사이즈 부족 등이 있다고 판단하였습니다. 이에 따라 모든 입출력 스트림과 문자열 변환 과정에서 반드시 "UTF-8" 인코딩을 명시하여 처리하였으나, 여전히 한글이 정상적으로 전송·수신되지 않는 문제가 완전히 해결되지는 않았습니다.

2. 등록 서버 포트와 채팅 포트 충돌 문제가 발생

등록 서버(UDP)와 채팅 기능(TCP)이 동일한 포트에서 동작할 경우, 포트 충돌로 인해 정상적

인 통신이 불가능한 문제가 발생하였습니다. 이를 해결하기 위해 등록 서버와 채팅 각각의 기능에 대해 별도의 포트를 사용하도록 설계하였으며, 사용자가 클라이언트 실행 시 등록 서버 포트와 채팅 포트 번호를 각각 직접 입력할 수 있도록 구현하였습니다. 또한, 클라이언트가 자신의 채팅 포트 번호를 등록 서버에 별도로 등록하여, 상대방이 채팅 연결 요청 시 올바른 포트로 접속할 수 있도록 하였습니다.

3. 유저 클라이언트 단일 구조로 인한 문제 발생

유저 클라이언트를 하나의 프로그램에서 등록 서버에 대한 클라이언트 기능과 채팅을 위한 서버-클라이언트 기능을 모두 처리하는 단일 구조로 구현하면서 여러 가지 문제가 발생하였습니다.

첫 번째로, 채팅 요청 시 타 클라이언트가 해당 요청을 실시간으로 수신할 수 있도록 하는 구조가 필요했습니다. 이를 해결하기 위해, 각 클라이언트에서 채팅 수신 전용 쓰레드를 별도로 생성하여 항상 백그라운드에서 실행되도록 설계하였습니다. 이 방식으로 사용자가 등록 서버와 연동 기능을 사용하면서도 외부에서 들어오는 채팅 요청을 즉시 수신하고 응답할 수 있게 되었습니다.

두 번째로, 채팅 연결 이후에도 기존 등록 서버와 연동하는 메뉴가 계속 반복 실행되어, 채팅 도중 입력이 꼬이거나 메뉴가 중복 출력되는 문제가 발생하였습니다. 이 문제는 채팅 중 상태를 나타내는 isChatting 변수를 도입해, 채팅 중에는 메뉴 루프가 일시 정지되고 채팅 종료 후에만 메뉴로 복귀하도록 개선하여 해결하였습니다. 추가로, 버퍼 꼬임 현상은 메뉴에 새로 진입할 때 입력 버퍼를 초기화하는 방식을 적용하여 보완하였습니다.

마지막으로, 채팅 시작과 종료 시 입력 버퍼가 꼬이거나 동일한 안내 문구가 중복 출력되는 현상이 발생하였습니다. 버퍼 꼬임 문제의 경우, 완벽한 기술적 해결책을 찾기 어려웠기 때문에, “엔터를 입력 후 채팅을 시작하세요” 또는 “엔터를 입력하면 메뉴로 돌아갑니다”와 같은 안내 문구를 명확히 삽입하였고, 판정 변수에 빈 문자열(“”)이 입력된 경우 적절히 다음 처리가 이루어지도록 설계하였습니다. 또한, 동일한 안내 문구가 중복 출력되는 현상은 채팅 기능의 각 스레드별 종료 처리가 별도로 이뤄지기 때문에 발생하는 문제로 확인되었습니다. 이 부분은 채팅 상태를 나타내는 isChatting 변수를 활용하여, 이미 종료된 상황에서는 추가 안내가 출력되지 않도록 조건을 두어 중복 출력을 방지하였습니다.

3. 연구과제 구현방법

연구과제내용

아래와 같은 연구과제 사항이 있었습니다.

1. 클라이언트들의 이동성 구현 설계 및 방안

다양한 네트워크 환경(예: 이동형 디바이스, 변동하는 IP/포트 등)에서도 클라이언트가 자유롭게 서비스에 접근할 수 있도록 지원하는 것은 분산 네트워크 시스템에서 매우 중요한 요구사항

항입니다. 본 프로젝트에서는 이러한 요구를 충족하기 위해, 클라이언트가 자신의 접속 정보(IP, 포트, ID 등)를 직접 등록 서버에 등록, 수정, 삭제할 수 있는 구조를 설계하였습니다.

최초의 등록의 경우 아이디를 입력하면 자동으로 클라이언트의 IP와 포트 정보가 등록 서버로 전송되고, 등록됩니다. 이후 네트워크 환경이 바뀌거나 포트가 변경되는 경우, 자신의 아이디를 입력하여 ‘수정(업데이트)’ 기능을 통해 자신의 정보를 최신 상태로 갱신할 수 있으며, 서비스를 더 이상 필요하지 않은 경우 ‘삭제’ 기능을 이용해 서버에서 자신의 정보를 제거할 수 있습니다.

이를 통해 각 클라이언트는 수동으로 자신의 네트워크 정보를 최신 상태로 유지하며, 다양한 네트워크 환경 변화에도 안정적으로 서비스를 이용할 수 있습니다. 현재 구현된 코드는 아이디만 입력하면 타인의 정보도 수정할 수 있는 한계가 있지만, 비밀번호 또는 인증 시스템을 도입하여 오직 본인만 자신의 정보를 수정·삭제할 수 있도록 확장해보고 싶습니다.

2. 소켓 기반 서비스 전송

네트워크 구조는 등록 서버와 클라이언트 간의 통신은 UDP 소켓을 기반으로 하여, 사용자 등록, 조회, 수정, 삭제 등 관리 메시지를 빠르게 주고받을 수 있도록 설계하였습니다. 이후 클라이언트 간 채팅 기능은 신뢰성과 데이터 순서 보장을 위해 TCP 소켓을 사용하여 구현하였습니다.

3. 프로토콜 구조

클라이언트와 등록 서버 간의 통신은 간결한 문자열 기반 메시지 포맷(예: Register,ID,포트 번호 또는 Query_User,ID)을 사용합니다. 각 명령은 구분자(침표)로 나누어 해석하기 쉽게 만들었으며, 등록·조회·수정·삭제·채팅 연결 등 기능별로 메시지 형식을 표준화하였습니다.

각 클라이언트는 언제든지 자신의 정보를 서버에 등록·수정할 수 있고, 새로운 사용자가 추가되어도 기존 구조에 영향을 주지 않도록 설계하였습니다.

클라이언트 간 채팅의 경우, 사용자가 메뉴에서 “사용자와 채팅 시작”을 선택하면, 먼저 등록 서버에 접속해 상대방의 최신 연결 정보를 조회합니다. 이후 조회된 IP와 포트 정보를 바탕으로 직접 TCP 통신을 통해 상대방에게 채팅 연결을 요청하게 됩니다. 이때 각 클라이언트는 채팅 연결을 수신할 수 있도록 항상 별도의 쓰레드에서 연결 대기 상태를 유지하게 하여, 실시간 채팅 서비스가 가능하도록 구현하였습니다.

구현 방법

1. 클라이언트들의 이동성 구현 설계 및 방안

먼저, 사용자 등록을 관리하는 등록 서버(RegisterServer.java)를 구현하였습니다. 이 서버는 기존 echoserver1.java 코드를 참고하여, 기본적으로 7번 포트에서 동작하도록 설계되었습니다. 네트워크 통신에는 MyServerDatagramSocket.java를 활용하여 UDP 기반으로 포트를 개방하였습니다. 서버는 HashMap을 이용해 사용자 정보를 효율적으로 저장 및 관리합니다.

서버의 동작 구조는 다음과 같습니다.

- 1) 서버는 데이터 받기 대기 상태의 루프에 들어갑니다.
- 2) 클라이언트에서 메시지를 전달받으면, 이를 구분자 “,” 단위로 나누어 명령어와 사용자 정보를 구분합니다.
- 3) 분리된 명령어에 따라 등록, 조회, 수정, 삭제 등 해당 작업을 수행한 후, 그 결과를 다시 클라이언트에 전송합니다.

이러한 등록 서버 구조를 통해, 각 클라이언트는 자신의 정보를 직접 등록, 수정, 삭제할 수 있고 네트워크 환경이 변화(예: 포트 변경, IP 변경 등)하더라도 본인의 최신 정보를 쉽게 갱신할 수 있습니다. 이로써, 클라이언트 이동성을 실질적으로 지원할 수 있도록 구현하였습니다. 다만 아이디만을 이용하여 정보를 갱신하는 방식을 사용하여 타인의 정보도 수정할 수 있다는 점은 추후 수정해보고 싶은 과제입니다.

2. 소켓 기반 서비스 전송

등록 서버는 위에서 서술한 바와 같이 MyServerDatagramSocket.java 코드를 기반으로 UDP 소켓을 사용하였습니다. 유저 클라이언트는 등록 서버와 통신하기 위해 UdpHelper.java 파일을 사용하며 이는 클래스 파일로, EchoClient1Helper.java 실습코드를 참고하여 작성하였습니다. UdpHelper 코드 내부에서는, 등록 서버와의 UDP 통신을 위해 MyClientDatagramSocket.java 코드를 활용하였습니다. 이를 통해 클라이언트와 서버 간의 메시지 송수신이 효율적으로 이루어질 수 있도록 설계하였습니다.

클라이언트간 통신을 위해 메시지 송신용 클래스, TcpHelper.java와 메시지 수신용 클래스, TcpReceiverHelper.java 두 가지 클래스를 구현하였습니다. 이 중 TcpHelper.java는 EchoClient2.java 실습 코드를 , TcpReceiverHelper.java는 EchoServer3.java 실습 코드를 참고하여 작성하였습니다.

특히 TcpReceiverHelper.java 내부적으로 TcpReceiverThread.java라는 하위 클래스를 가지며, 이는 EchoServerThread.java 실습코드를 참고하여 작성하였습니다. 이러한 코드들은 MyStreamSocket.java 파일을 활용하여 TCP 소켓 구조를 기반으로 클라이언트 간 안정적인 통신이 가능하도록 구현하였습니다.

3. 프로토콜 구조

프로토콜의 구조는 크게 2가지로 구분됩니다. 클라이언트와 등록 서버간의 통신, 그리고 클라이언트와 클라이언트간의 통신입니다. 프로그램의 실행 과정의 캡처와 함께 설명하도록 하겠습니다.

```
채팅 시스템에 어서오세요.
사용자 등록을 위해 등록 서버에 연결하여야 합니다.
서버의 주소를 입력하여 주세요 (엔터시 localhost):
서버 포트 번호를 입력해주세요. (엔터시 7):
채팅 수신 포트 번호를 입력하세요: 1000
```

상대방의 채팅 연결을 기다리는 중...

```
==== 메뉴 ====
1. 사용자 등록
2. 전체 사용자 조회
3. 사용자 정보 삭제
4. 사용자 정보 수정(재등록)
5. 사용자와 채팅 시작
6. 프로그램 종료
=====
```

위 화면은 유저 클라이언트를 실행했을시 뜨는 기본 UI입니다. 우선 사용자는 등록 서버와 연결을 위해 등록 서버의 주소와 포트번호를 입력해야합니다. 이 값은 기본값이 설정되어있으며, 각각 localhost와 7번 포트입니다. 이후 채팅 시스템을 이용하기 위한 채팅 수신용 포트번호를 설정해야 합니다. 이 값은 필수이기 때문에 입력하지 않으면 프로그램이 동작하지 않습니다. 등록 서버와 통신 메뉴가 뜨기 전 우선 TCP 연결 요청을 받기 위해 백그라운드에서 동작하는 Tcp 수신 쓰레드가 실행되며 초기 설정이 완료됩니다.

먼저 클라이언트와 등록 서버 간의 통신과 프로그램 작동에 대해 설명하겠습니다. 아래는 각 메뉴의 기능별 동작 구조를 캡처 화면과 함께 설명하겠습니다.

```
번호를 선택하세요: 1
등록할 사용자 ID를 입력하세요: user1
서버 응답: ID registration complete,user1
```

클라이언트 명령

```
Server ready.
Request received.
Received message: Register,user1,1000
user1 Registered: 127.0.0.1,1000
```

서버 동작

```
번호를 선택하세요: 1
등록할 사용자 ID를 입력하세요: user1
서버 응답: ERROR,AlreadyRegisteredUser
Received message: Register,user1,1000
중복 등록 거부: 127.0.0.1,1000
```

이미 등록된 사용자의 경우

- 사용자 등록은 "Register,<ID>,<Port>" 형태로 클라이언트가 자신의 ID와 채팅용 포트번호를 등록 서버에 전달합니다. 이후, 등록 서버는 "ID registration complete, id"의 응답을 보내 등록이 완료되었음을 전달합니다. 또는 이미 등록된 사용자의 경우에는 "ERROR, Already Registered User" 응답을 전달합니다.


```
번호를 선택하세요: 2
[서버 응답:
USER LIST
1. user1
```

```
Request received.
Received message: Query
```

- 전체 사용자 조회는 클라이언트가 "Query" 메시지로 현재 등록된 모든 유저 목록 요청합니다. 이후, 등록 서버는 전체 사용자의 목록을 클라이언트로 전달합니다. 위 캡처는 각각 클라이언트와 서버의 출력입니다.

```
번호를 선택하세요: 3
등록할 사용자 ID를 입력하세요: user1
[서버 응답: Deleted,user1
```

```
Request received.
Received message: Delete,user1
```

```
번호를 선택하세요: 2
[서버 응답: No connected users.
```

삭제 이후 유저 목록

- 사용자 정보 삭제는 "Delete,<ID>" 형태로 데이터를 보내 해당 ID의 사용자 등록정보 삭제 요청합니다. 이후 해당 아이디의 데이터는 등록 서버에서 삭제 됩니다.

```
번호를 선택하세요: 4
등록할 사용자 ID를 입력하세요: user1
[서버 응답: UPDATED,user1
```

```
Request received.
Received message: Update,user1,1000
```

사용자 정보 수정(재등록)은 "Update,<ID>,<Port>"의 형태로 클라이언트의 접속 정보를 최신 정보로 수정 요청 메시지를 보내면 응답 서버 내에서 해당 아이디를 가진 유저의 포트 정보와 아이피 정보를 수정하여 저장합니다.

```
번호를 선택하세요: 5
채팅할 상대방 ID를 입력하세요: user2
상대방 정보: IP=127.0.0.1, Port=1001
[Connection request made
```

```
Request received.
Received message: Query_User,user2
```

- 특정 사용자 정보 조회는 채팅 연결을 요청할 시 클라이언트가 "Query_User,<ID>" 형태로 해당 ID의 사용자 IP/Port 정보 요청하며 응답 서버는 돌려받은 정보를 바탕으로 클라이언트 끼리 통신을 실행합니다.

지금까지의 내용은 등록 서버와 유저 클라이언트간의 상호작용에 대한 설명이었습니다. 다음으로 유저 클라이언트간 통신에 대해 설명하겠습니다.

```
번호를 선택하세요:
채팅이 연결되었습니다. 엔터를 한번 누른 후 채팅을 입력해주세요.

hello
[상대] hello
```

우선 채팅을 연결 받은 쪽입니다. 채팅을 연결 받으면 기존에 메뉴 응답 버퍼가 꼬이기 때문에 엔터를 한번 눌러주는 것으로 처리하였습니다. 엔터를 누른 이후 통신을 시작하면 정상적

으로 통신을 시작하며 메뉴의 경우 채팅 체크 변수가 true로 돌아가 무한 대기 상태로 전환됩니다.

```
Connection request made
상대방과 채팅을 시작합니다. (exit 입력시 종료)
[상대] hello
hello
_
```

통신을 걸은 쪽의 경우에는 기존의 메뉴 안에서 Tcp 통신이 종료될 때까지 메시지를 보내는 코드가 무한히 돌아가고 있기 때문에 메뉴를 보류 처리하지 않아도 정상적으로 동작하는 모습을 확인할 수 있습니다.

```
[상대] hello
exit
연결 종료

==== 메뉴 ====
1. 사용자 등록
2. 전체 사용자 조회
3. 사용자 정보 삭제
4. 사용자 정보 수정(재등록)
5. 사용자와 채팅 시작
6. 프로그램 종료
=====

번호를 선택하세요:
상대방의 채팅 연결을 기다리는 중...
```

이후 통신의 받은쪽에서 exit 명령어를 사용하여 연결을 종료한 경우 정상적으로 연결 종료 후 정상적으로 메뉴로 돌아오는 것을 확인할 수 있습니다. 이후 재통신을 위해 상대방의 채팅 연결을 기다리는 백그라운드 쓰레드가 동작하는 것을 확인할 수 있습니다.

```
[상대] hello
hello
상대방과의 연결이 종료되었습니다.1 엔터를 입력하면 메뉴로 돌아갑니다.

==== 메뉴 ====
1. 사용자 등록
2. 전체 사용자 조회
3. 사용자 정보 삭제
4. 사용자 정보 수정(재등록)
5. 사용자와 채팅 시작
6. 프로그램 종료
=====

번호를 선택하세요: _
```

통신을 건 쪽의 경우 상대방에게 보내기 위한 버퍼가 남아있기 때문에, 엔터를 한번 입력해주어야 메뉴로 돌아가는

것을 확인할 수 있습니다. 기존에 이 과정을 구현하면서 많은 버퍼 꼬임 현상이 있었지만, 채팅 중인지 확인하는 변수를 통하여 중복되는 출력 및 버퍼를 동작을 멈추는 방식을 사용하였고 기존에 입력중인 버퍼는 입력을 해야만 다음으로 넘어가기 때문에 엔터를 입력하여 정상 작동하도록 설계하였습니다.

```
채팅할 상대방 ID를 입력하세요: user2
상대방 정보: IP=127.0.0.1, Port=1001
Connection request made
상대방과 채팅을 시작합니다. (exit 입력시 종료)
[상대] hello
hello
exit
상대방과의 연결이 종료되었습니다.3

==== 메뉴 ====
1. 사용자 등록
2. 전체 사용자 조회
3. 사용자 정보 삭제
4. 사용자 정보 수정(재등록)
5. 사용자와 채팅 시작
6. 프로그램 종료
=====

번호를 선택하세요:
```

마찬가지로 채팅을 건쪽에서 종료시에도 정상적으로 메뉴로 넘어가는 것을 확인할 수 있습니다.

```
채팅이 연결되었습니다. 엔터를 한번 누른 후 채팅을 입력해주세요.

hello
[상대] hello
[상대] exit
상대방이 채팅을 종료했습니다. 엔터를 입력하면 메뉴로 돌아갑니다.1

상대방의 채팅 연결을 기다리는 중...

==== 메뉴 ====
1. 사용자 등록
2. 전체 사용자 조회
3. 사용자 정보 삭제
4. 사용자 정보 수정(재등록)
5. 사용자와 채팅 시작
6. 프로그램 종료
=====

번호를 선택하세요:
```

채팅 종료를 받은 쪽에서도 위의 상황과 마찬가지로 버퍼가 남아있기 때문에 엔터를 눌러 정상적인 메뉴로 넘어가는 방식으로 구현하였습니다.

III. 결론

1. 연구 결과

본 프로젝트에서는 등록 서버와 클라이언트 간의 통신, 그리고 클라이언트 간 직접 통신(채팅) 기능을 모두 갖춘 분산 채팅 시스템을 구현하였습니다. 구현 과정에서 다음과 같은 핵심 알고리즘 및 설계적 특징을 적용하였습니다.

등록 서버와의 통신은 UDP 기반의 문자열 명령 프로토콜을 사용하여, 사용자의 정보 등록, 수정, 삭제, 조회 등의 기능을 신속하게 처리하였습니다. 각 명령은 콤마(,)로 구분되는 간결한 포맷을 사용하여 네트워크 상에서의 데이터 처리를 단순화하였습니다.

클라이언트 간 채팅은 TCP 소켓을 이용해 신뢰성 있는 데이터 전송을 보장하였고, 각 클라이언트는 별도의 채팅 수신 쓰레드를 항상 실행함으로써 외부에서의 연결 요청에도 실시간으로 대응할 수 있도록 하였습니다. 클라이언트 이동성 지원을 위해 각 클라이언트가 자신의 네트워크 정보를 직접 등록, 수정, 삭제할 수 있도록 하여, 네트워크 환경 변화(포트 변경 등)에 능동적으로 대처할 수 있도록 설계하였습니다.

버퍼 꼬임/동시성 문제 해결을 위해, 채팅 중에는 메뉴 입력 루프를 일시정지(isChatting 변수 활용)시키고, 채팅 종료 후에만 정상적으로 메뉴로 복귀하도록 하였습니다. 또한, 엔터키 입력요청 문구를 작성하여 완벽히 개선하진 못하더라도 버퍼 꼬임 문제를 최소화하였습니다.

2. 프로그램 평가 및 보완점

본 프로그램은 분산 환경에서 기본적인 사용자 등록, 실시간 채팅 기능, 그리고 클라이언트 이동성을 모두 검증하였습니다. 다만, 한글 메시지 전송의 인코딩 문제, 동시다발적 요청 처리(여러 요청이 동시에 오는 경우 문제 발생), 인증(비밀번호 등) 기능, 서버가 켜지지 않은 상태에서 통신시 데드락 걸림 현상 등은 많은 부분에서 오류가 발생하며 추가적인 개선이 필요하지만, 프로젝트를 완성하는 기한과 저의 역량 부족으로 전부 개선하지는 못하였습니다. 이외에도 부족한 부분이 많고, 실제 다른 네트워크 상에서 동작한 것이 아닌 로컬 호스트상에서만 작동 테스트를 하여 발견하지 못한 오류도 많을 것이라 생각합니다. 추후 이러한 부분을 과제로서가 아닌, 개인의 역량 강화를 위해 추후 개선하여보고 싶다고 생각합니다.

3. 평가자를 위한 참고 사항

각 기능의 수행 흐름과 주요 알고리즘은 본 보고서의 코드 설명 및 주석을 작성하였습니다. 프로그램 실행 및 테스트 시, 등록 서버(UDP)를 각 클라이언트의 채팅 포트(TCP)를 반드시 별도로 실행해야 정상 동작합니다. 또한, 서버를 켜놓지 않은 상태에서 클라이언트 동작시 무한 대기가 발생합니다. 채팅 중에는 엔터 입력 등으로 버퍼 꼬임 현상이 있을 수 있으니, 안내 메시지를 참고하여 테스트를 진행해 주시면 감사드립니다.